

category: camera-integration

tags: [kztek-cameras, avalonia, skbitmap, memory-leak, event, dead-subscriber, ffmpeg, lpr, motion-detection]

severity: critical

created: 2026-07-20

updated: 2026-07-20

project-origin: parking-v8-app-avalonia

‘AnvPlayerService’ cấp phát ‘bmp.Copy()’ cho event KHÔNG CÓ subscriber nào → rò rỉ native memory liên tục khi camera thật đang chạy

Tình huống gặp phải

User báo: "khi không có camera, app không tăng RAM; khi có camera thật kết nối, RAM tăng rất nhanh và không giảm". Đây chính là kịch bản "Pending verification với camera thật" còn treo lại từ BUG-004 ([docs/bugs/BUG-004-mainshell-freeze-after-login.md](#)) — round trước chỉ test được với camera KHÔNG kết nối (dev machine không có camera IP thật).

Nguyên nhân gốc rễ (Root Cause)

`AnvPlayerService.ConsumerMotionDetectionAsync()` và `ConsumerAiDetectionAsync()` (trong `Kztek.Cameras.Avalonia`, BaseLIB — dùng chung bởi nhiều project KZTEK) mỗi khi phát hiện biển số mới (không trùng lặp) hoặc AI box đổi trạng thái, đều:

```
var vehicleImageForEvent = bmp.Copy(); // SKBitmap MỚI, KHÔNG qua SKBitmapPool – sở hữu riêng
var data = new MotionAnalyzingEventArgs(conf, vehicleImageForEvent, plate.Crop, plate.Text);
_ = Task.Run(() => MotionAnalyzing?.Invoke(this, data), token);
```

`bmp.Copy()` cấp phát native SKBitmap **hoàn toàn mới** (khác với `_currentSKFrame` — không đi qua `SKBitmapPool`, xem lesson [kztek-cameras-avalonia-getframe-returns-skbitmap.md](#)). Grep toàn bộ `src/` của `parking-v8-app-avalonia` xác nhận: **KHÔNG có bất kỳ subscriber nào** cho 3 event `MotionAnalyzing`, `AI DetectAnalyzing`, `MotionTriggerDetected` trong toàn bộ app — `CameraView.axaml.cs` chỉ forward các event này từ `_service` lên `AnvPlayer` public event, nhưng không có `ViewModel/View` nào ở tầng app đăng ký nghe.

Hệ quả: mỗi khi camera thật hoạt động và có xe đi qua (motion trigger → LPR nhận diện biển số mới, hoặc AI box đổi trạng thái Open/Close), code cấp phát 1 SKBitmap full-frame, gói vào event args, bắn event vào "khoảng không" (`?.Invoke` trên delegate null = no-op), rồi **không có reference nào giữ lại và cũng không Dispose** — native memory (Skia) rò rỉ vĩnh viễn, chỉ được GC finalizer dọn (không đáng tin, chậm, dưới áp lực camera decode liên tục để tích lũy nhanh).

Vì môi trường dev không có camera IP thật, motion/LPR pipeline chưa từng thực sự chạy → leak này KHÔNG xuất hiện khi test với camera giả/offline (khớp chính xác với báo cáo "không camera → không tăng RAM").

Giải pháp

Dispose bitmap tự cấp phát ngay sau khi event đã fire xong (không phụ thuộc có subscriber hay không):

```
// MotionAnalyzing (Task.Run - async, dispose trong finally của cùng task):
var vehicleImageForEvent = bmp.Copy();
var data = new MotionAnalyzingEventArgs(conf, vehicleImageForEvent, plate.Crop,
plate.Text);
_ = Task.Run(() =>
{
    try { MotionAnalyzing?.Invoke(this, data); }
    finally { vehicleImageForEvent.Dispose(); }
}, token);

// AIDetectAnalyzing (gọi Invoke đồng bộ - dispose ngay sau Invoke trong try/finally):
var fullFrameClone = bmp.Copy();
var boxedFrame = DrawRect(fullFrameClone, ...); // vẽ in-place, trả về CÙNG bitmap
try { AIDetectAnalyzing?.Invoke(this, args, boxedFrame); }
finally { boxedFrame.Dispose(); }
```

File:

Kztek.Camera/1.Source/Kztek.Cameras.Avalonia/Players/FFMPEG/UserControls/AnvPlayerService.cs.

Áp dụng lại (How to reuse)

- Bất kỳ event nào truyền `SKBitmap/Bitmap`/tài nguyên native qua `EventArgs` mà code publisher tự cấp phát (`bmp.Copy()`, `new SKBitmap(...)`, không qua pool) — PHẢI tự dispose sau khi `Invoke()` hoàn tất, KHÔNG được ỷ lại subscriber dispose giùm, vì:
 - Event có thể không có subscriber nào (dead event) — vẫn cấp phát native memory mỗi lần fire.
 - Ngay cả khi có subscriber, hợp đồng "ai dispose" thường không rõ ràng trừ khi ghi chú tường minh trong doc comment.
- Khi điều tra memory leak "chỉ xảy ra khi có tín hiệu/dữ liệu thật, không xảy ra ở trạng thái idle/giả lập" → nghi ngờ ngay các pipeline chỉ chạy khi có sự kiện thật (motion trigger, LPR match, AI detect) — đây thường là nơi test ở dev/không camera không bao giờ chạm tới.
- Cách xác minh nhanh:** `grep -rn "EventName\b" src --include=*.cs` ở tầng app — nếu event có publisher (BaseLIB) nhưng 0 kết quả subscriber ở app layer, MỌI bitmap/tài nguyên cấp phát cho event đó là leak chắc chắn, không cần profiler để xác nhận.

CẬP NHẬT QUAN TRỌNG (2026-07-20, sau khi test với camera thật): đây KHÔNG phải leak chính

Sau khi áp dụng fix trên, user xác nhận RAM **vẫn tăng ~35-80MB/s** kể cả khi camera KHÔNG chạy chế độ motion (`VirtualLoopType.None`). Instrument tạm `SKBitmapPool.Rent()` để log song song `GC.GetTotalMemory(false)` (managed) vs `Process.WorkingSet64/PrivateMemorySize64` (native) cho kết quả **quyết định**:

rents	GC_MB (managed)	Priv_MB (native)
-------	-----------------	------------------

---	---	---
50	32.4	399.8
400	26.6	1043.5
700	28.1	1755.1

Managed heap gần như phẳng, `SKBitmapPool` chỉ có 2 key cố định + 6 lần alloc TỔNG (hoàn toàn ổn định) — trong khi native/private memory tăng >1.3GB cùng lúc. **Kết luận: leak thật sự nằm ở tầng P/Invoke FFmpeg native (VideoStreamDecoderIntptr), KHÔNG phải ở SKBitmap/event/managed layer** như lesson này ban đầu kết luận. Fix `bmp.Copy()` / `plate.Crop` ở trên vẫn ĐÚNG và nên giữ (leak thật, chỉ là không phải nguồn chính), nhưng KHÔNG giải quyết được phần lớn RAM tăng khi camera thật đang stream.

Bài học phương pháp quan trọng: Khi nghi ngờ memory leak trong app C#/.NET, đo

`GC.GetTotalMemory(false)` SONG SONG với `Process.WorkingSet64/PrivateMemorySize64` NGAY TỪ ĐẦU trước khi đi sâu vào bất kỳ tầng managed nào (`SKBitmap`, `List`, `Dictionary`...) — nếu managed heap phẳng mà process memory vẫn tăng, toàn bộ thời gian debug ở tầng C#/managed là lãng phí, phải chuyển hướng ngay sang tầng native/P-Invoke (cần profiler native: `VMMMap`, `Process Hacker`, `WinDbg` — KHÔNG thể debug tiếp chỉ bằng đọc code C#). Chi tiết đầy đủ: [docs/bugs/BUG-004-mainshell-freeze-after-login.md](#) mục "Root cause vòng 4".

CẬP NHẬT 2 (2026-07-20): tìm ra + giảm được ~50% qua thực nghiệm — 2/3 camera kệt decode full-res vĩnh viễn

Instrument log phân biệt theo từng `AnvPlayerService` instance (`svc=<hashcode>`) phát hiện: trong 3 camera cùng chạy, 2/3 camera VĨNH VIỄN có `Bounds=(0,0)` (không phải dao động ngẫu nhiên) — vì các tile này nằm trong lane/tab KHÔNG được chọn hiển thị, và Avalonia `TabControl` KHÔNG BAO GIỜ Measure/Arrange nội dung tab chưa chọn (lazy layout — xem [avalonia-tabcontrol-lazy-visual-tree-columns.md](#)). Guard "controlReady" trong `AnvPlayerService.PollingDecodeFrameAsync` chỉ tính cho race điều kiện lúc khởi động, không lường trước trường hợp tile KHÔNG BAO GIỜ hiển thị — khiến 2 camera này decode ở độ phân giải NATIVE (1280×720) liên tục vô thời hạn.

Fix: cap độ phân giải fallback khi `!controlReady` xuống tối đa 640×360 (giữ aspect ratio) thay vì native. Đo trước/sau với camera thật: ~35-40MB/s → ~18-19MB/s (giảm ~50%).

Quan trọng: phần còn lại (~18-19MB/s) KHÔNG giảm tỉ lệ thuận theo pixel (giảm 4× pixel chỉ giảm ~2× tốc độ) → còn 1 leak "cố định/frame" khác ở tầng FFmpeg native, chưa xác định được nguồn chính xác — cần `VMMMap`/`WinDbg`. Chi tiết đầy đủ: [docs/bugs/BUG-004-mainshell-freeze-after-login.md](#) mục "Root cause vòng 5".

Bài học phương pháp: Khi debug leak trong multi-instance pipeline (nhiều camera/service cùng chạy), PHẢI gắn ID phân biệt từng instance vào log (`GetHashCode()` hoặc tên) trước khi kết luận "dao động" hay "không ổn định" — nếu không, log interleaved từ nhiều instance dễ bị hiểu nhầm thành 1 instance đang flip-flop.

CẬP NHẬT 3 (2026-07-20): cô lập bằng test harness riêng — loại trừ HẢN tầng SKBitmap/managed

Tạo test harness độc lập: `CameraDemoWindow` (trong `ParkingV8.DemoHost`) fix cứng sẵn 1 camera test bằng video file cục bộ + tự động Connect (không cần login/multi-lane/nhập tay) — dùng lại được cho lần sau. Test với CHỈ 1 camera, Bounds hợp lệ ngay từ đầu, không motion: **RAM vẫn tăng ~7.4MB/s**, loại trừ hản:

nhiều camera, tab ẩn/Bounds=0, motion/LPR, và 2 fix trước đó.

Bằng chứng quyết định: instrument `SKBitmapPool.Rent()` log địa chỉ `bmp.GetPixels()` mỗi lần dequeue lại — 99/100 lần REUSE với địa chỉ **hoàn toàn ổn định** (0 lần đổi địa chỉ). SKBitmap pool được tái sử dụng đúng 100%, KHÔNG hề leak. → **Leak nằm hoàn toàn trong native FFmpeg** (`av_frame_unref`/buffer nội

bộ `AVFrame`), ngoài khả năng debug tiếp bằng đọc code C# — cần VMMap/Process Hacker/WinDbg.

Bài học phương pháp (quan trọng nhất của cả chuỗi điều tra này): Khi nghi ngờ 1 tầng cụ thể (ở đây là SKBitmap pool) là nguồn leak, đừng chỉ nhìn số liệu tổng hợp (RAM tăng) — hãy ****đo trực tiếp bằng chứng sở hữu tài nguyên**** (ở đây: theo dõi địa chỉ con trỏ native của CHÍNH object nghi ngờ qua nhiều lần dùng lại). Đây là cách nhanh nhất để CHỨNG MINH (không phải suy đoán) một tầng có phải nguồn leak hay không, mà không cần profiler ngoài.

Chi tiết đầy đủ: <docs/bugs/BUG-004-mainshell-freeze-after-login.md> mục "Root cause vòng 6".

CẬP NHẬT 4 (2026-07-20) — XÁC NHẬN DỨT KHOÁT: leak nằm 100% trong SkiaSharp SKBitmap, không phải FFmpeg

Test quyết định: thêm nhánh convert tạm bằng `System.Drawing.Bitmap + LockBits/UnlockBits` (y hệt WinForms)

NGAY TRONG CÙNG process Avalonia, dùng chung `sws_scale`, cùng file test — bitmap tạo ra discard ngay (không render). **Kết quả: RAM ổn định tuyệt đối ở 164MB trong 20s (không tăng 1 byte)**, so với ~7MB/s

khi dùng SKBitmap bình thường. Điều này loại trừ HÃN: FFmpeg, `sws_scale`, FFmpegLoader, decode loop, render

Avalonia — chỉ còn lại đúng 1 biến số: **bản thân kiểu `SKBitmap` (SkiaSharp 4.150.1)**.

Thử hạ version SkiaSharp để cô lập thêm — KHÔNG hạ được: `ReadPL 1.0.8` ép floor `>=4.150.0`,

`Kztek.Object.MultyPlatform` ép floor `>=4.150.1`. Version bị pin cứng toàn solution, không test được.

Kết luận cuối cùng của lesson này: Leak là native, nằm trong cách `SKBitmapPool.Rent()` →

`bmp.GetPixels()` (ghi trực tiếp qua `sws_scale`) → `bmp.NotifyPixelsChanged()` → `SKBitmapPool.Return()`

tương tác với SkiaSharp 4.150.1 khi TÁI SỬ DỤNG cùng 1 object nhiều lần, kết hợp `SKAlphaType.Opaque` (tổ hợp không phổ biến). Đây có thể là edge-case/regression riêng của SkiaSharp 4.150.1 — vượt quá khả năng sửa chỉ bằng code C# của app. Hướng tiếp theo: thử API SkiaSharp khác (`SKPixmap`/`SKSurface` thay `GetPixels()` trực tiếp), báo bug lên SkiaSharp GitHub kèm repro tối thiểu, hoặc profiler native để xác nhận call stack chính xác trong `libSkiaSharp.dll`.

Bài học phương pháp lớn nhất của cả chuỗi 12 vòng điều tra: Khi 1 module bị nghi leak nhưng không loại trừ được bằng cách tắt từng phần bên trong nó, hãy ****thay THẮNG module đó bằng 1 implementation khác đã biết là KHÔNG leak**** (ở đây: WinForms GDI+ Bitmap, đã biết chắc chắn ổn định), chạy NGAY TRONG

CÙNG process/pipeline/dữ liệu — đây là cách nhanh nhất và chắc chắn nhất để cô lập "module X có leak

hay không" mà không cần profiler, khi đã hết các giả thuyết rẻ tiền khác.

Chi tiết đầy đủ: [docs/bugs/BUG-004-mainshell-freeze-after-login.md](#) mục "Root cause vòng 11-12".

CẬP NHẬT 5 (2026-07-20) — FIX ÁP DỤNG: bypass SKBitmap bằng Avalonia WriteableBitmap (C2), verify hết leak

Sau khi xác nhận SKBitmap (SkiaSharp 4.150.1) là nguồn leak (cập nhật 4), đã áp dụng fix thực tế: đổi TOÀN BỘ pipeline decode (display + motion detection + AI box detection) từ `SKBitmap/SKBitmapPool` sang `Avalonia.Media.Imaging.WriteableBitmap/AvaloniaFramePool` (file mới) — ghi pixel qua `WriteableBitmap.Lock()` (cùng cơ chế con trỏ thô như GDI+ LockBits, đã test không leak) thay vì `SKBitmap.GetPixels()+NotifyPixelsChanged()`.

SKBitmap CHỈ còn tồn tại ở 2 điểm bắt buộc (API SDK ngoài yêu cầu, không đổi được):

`ReadPL.Recognize(SKBitmap)` và `MolexDetection.Predict(SKBitmap)` — cả 2 đều convert on-demand từ WriteableBitmap ngay tại điểm gọi (tần suất thấp — gated bởi `ShouldReadPlate/delayGetAI`, KHÔNG phải hot path 30fps đã gây leak).

Kết quả verify (`CameraDemoWindow`, 1 camera, ~40s liên tục): RAM từ tăng liên tục ~6-8MB/s → ổn định tuyệt đối 185-195MB, dao động bình thường. Build 0 Warning/0 Error.

Thay đổi hợp đồng quan trọng: `AnvPlayer.GetCurrentVideoFrame()/GetCurrentVideoFrame2()` trước đây

trả tham chiếu sống, CẤM dispose (xem cập nhật ở trên/lesson gốc) — giờ trả **BẢN SAO SKBitmap riêng, PHẢI dispose**. Đã cập nhật cả 3 call site: `CameraView.axaml.cs` (`CaptureSnapshotBitmapAsync/ProbeFirstFrameAsync`), `CameraFocusWindow.axaml.cs` (`PollFrameAsync`).

Phạm vi KHÔNG mở rộng (quyết định có lý do kỹ thuật, xem chi tiết trong BUG-004.md):

`ParkingV8.Lpr` (ILpr/AmericaLpr/Kzteklprs) dùng SKBitmap kiểu "cấp phát-dùng 1 lần-giải phóng" (snapshot → encode → HTTP → dispose), KHÔNG phải mẫu hình tái sử dụng lặp lại gây leak — đã xác nhận `EntryLaneLprService.cs` dispose đúng (`using var skImage = ...`). Code chết (`frmRegions/frmViewCamera/frmViewImage.axaml.cs, HwVideoStreamDecoder.cs`) không cần sửa vì không bao giờ chạy.

Bài học lớn nhất: Khi tìm ra 1 API/kiểu dữ liệu cụ thể là nguồn leak (ở đây: SKBitmap ghi qua `GetPixels` ở tần suất cao), đừng vội thay thế MỌI nơi dùng kiểu đó trong toàn bộ codebase — hãy phân biệt **mẫu hình sử dụng**: tái sử dụng lặp lại ở tần suất cao (hot path, cần sửa) vs cấp phát-dùng 1 lần-giải phóng (an toàn, không cần sửa dù cùng kiểu dữ liệu).

Chi tiết đầy đủ: [docs/bugs/BUG-004-mainshell-freeze-after-login.md](#) mục "FIX ÁP DỤNG (C2)".

CẬP NHẬT 6 (2026-07-20) — Sau khi đổi hợp đồng C2, PHẢI grep lại TOÀN BỘ call site cũ của `GetCurrentVideoFrame\*()`

Khi commit code camera lib (`Kztek.Camera BaseLIB`) sau fix C2, phát hiện `frmViewCamera.axaml.cs` (`OnTimerTick, DispatcherTimer 300ms`) vẫn gọi `_player.GetCurrentVideoFrame()` theo **hợp đồng CŨ** ("chỉ đọc Width/Height, KHÔNG dispose") — vì file này được sửa/thêm watchdog ở một nhánh công việc khác

(Milesight camera support + watchdog timeout) **song song cùng lúc** với fix C2, nên không được cập nhật theo hợp đồng mới. Do timer chạy mỗi 300ms, đây là 1 leak site thực sự (mỗi tick tạo 1 bản sao `SKBitmap` sở hữu riêng rồi bỏ rơi, không dispose) — chỉ bị phát hiện nhờ **đọc lại toàn bộ diff trước khi commit**, không phải nhờ test RAM (test trước đó không đi qua đường `frmViewCamera`, chỉ qua `CameraDemoWindow/ParkingV8.App`).

Bài học: Khi đổi hợp đồng ownership của 1 API dùng chung (dispose ↔ không dispose), KHÔNG đủ để chỉ sửa các call site đã biết từ trước — PHẢI Grep lại toàn bộ `GetCurrentVideoFrame` (hoặc tên hàm tương ứng) trong TOÀN BỘ solution/repo liên quan **ngay trước khi commit**, kể cả file đang được sửa song song bởi nhánh công việc khác trong cùng session hoặc bởi thay đổi chưa commit từ trước — vì các nhánh song song không tự động biết hợp đồng vừa đổi.

Cách áp dụng lại: Sau bất kỳ thay đổi ownership contract nào (dispose semantics), chạy

`grep -rn "TênHàm" --include=*.cs` trên toàn bộ solution/repo **TRƯỚC** khi commit, không chỉ dựa vào danh sách call site đã nhớ từ lúc bắt đầu task.

Chú ý / Cạm bẫy (Gotchas)

- ⚠️ Đừng nhầm với lesson `kztek-cameras-avalonia-getframe-returns-skbitmap.md` — đó là case NGƯỢC LẠI (KHÔNG được dispose vì là tham chiếu sống dùng chung). Ở đây `bmp.Copy()` là bản sao SỞ HỮU RIÊNG do chính dòng code đó tạo ra — PHẢI dispose. Luôn xác định rõ nguồn gốc bitmap (pool/live-reference vs `.Copy()/new SKBitmap()` độc lập) trước khi quyết định dispose hay không.
- ⚠️ `plate.Crop` (từ ReadPL SDK, native, không có source) KHÔNG được dụng vào (không thêm Dispose) — chưa có tài liệu xác nhận ownership, rủi ro double-free giống case `SKBitmapPool.Return()` đã từng gây `AccessViolationException` trước đây.
- ⚠️ BUG-004 (`docs/bugs/BUG-004-mainshell-freeze-after-login.md`) mô tả 6 fix trước đó chỉ verify được ở kịch bản KHÔNG có camera thật (offline/reconnect loop) — đây là fix thứ 7, phát sinh khi camera thật đã kết nối và có xe di chuyển, ngoài phạm vi các fix trước.

Tham chiếu

- `E:\KZTEK\Code_Git\1.Window\0.BaseLIB\Kztek.Camera\Kztek.Camera\1.Source\Kztek.Cameras.Avalonia\Players\FFMPEG\UserControls\AnvPlayerService.cs` — `ConsumerMotionDetectionAsync()` (~dòng 561-580), `ConsumerAiDetectionAsync()` (~dòng 681-699)
- `docs/bugs/BUG-004-mainshell-freeze-after-login.md`
- Project: `parking-v8-app-avalonia`, session 2026-07-20